7.1.6 Additional considerations for summarizing documents

There are two problems that frequently arise when summarizing documents. First, PDF semantic chunking is difficult. One challenge with PDF documents is that they often lack a clear separation into logical chunks or sections. Unlike web pages or word-processing documents, the content in PDFs may not have well-defined structural elements like headings, paragraphs, or sections. This can make it difficult to identify the most important parts of the document and extract relevant information for summarization. DOCX files, on the other hand, are easier to parse and can be grouped into logical sections more readily.

Then there's the fact that large prompts are slow and expensive for very large documents. When summarizing lengthy documents, the input prompt can become extremely large, often exceeding the token limits of LLMs. This can lead to increased processing times and higher computational costs. Techniques like MapReduce and refine can help, but they essentially involve looping over all chunks, which can still be inefficient for exceptionally large documents. To address some of the concerns about prompt size and cost effectiveness, we can explore two techniques that have shown good prospects.

Prompt compression can significantly reduce the length of the input prompt while preserving the essential information. There is active research into tools, such as LLMLingua, developed by Microsoft, that compress prompts. The main idea is to use for the compression process a small local LLM that is fast and uses different types of compression techniques to reduce the prompt size while maintaining the same prompt information. As illustrated in figure 7.6, this adds an intermediary step before sending the prompt to the LLM for processing.

By compressing the original prompt, the number of tokens is reduced, achieving 2.5× compression for standard prompts, up to 10× for conversation history, and 20× for highly repetitive content. This allows for faster turnaround times when processing the prompt through the LLM.

> **NOTE** *LLMLingua* is currently available as a plugin to the LangChain Python library and is not yet integrated into the LangChain.js framework.

Merging similar chunks together using k-mean clustering is another approach to representing the semantic content of large documents, such as document embeddings. These representations can then be used as the input for the summarization model rather than relying solely on the raw text. This can be particularly useful for summarizing long or complex documents where simply extracting and compressing the text may not be sufficient. The key steps in this process are

1. *Sentence splitting and chunking*—First we split and chunk the text using `RecursiveCharacterTextSplitter` since it is designed to divide large texts into manageable chunks while preserving semantic overlaps.